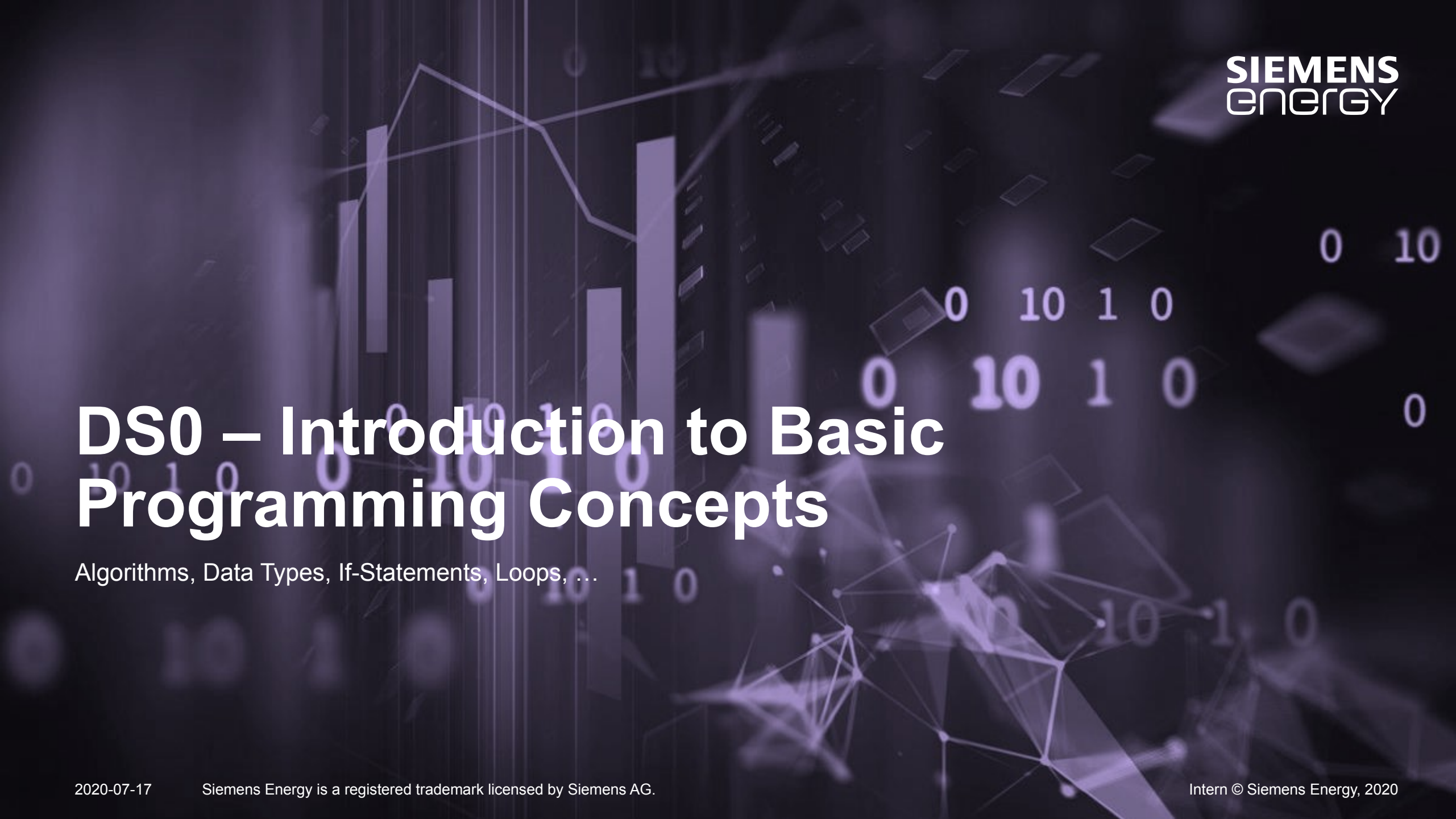


DS0 – Introduction to Basic Programming Concepts

Algorithms, Data Types, If-Statements, Loops, …

Agenda

1. Introduction: Programming & Algorithms
2. Data Types
3. Basics – Operators & Variables
4. Data Structures
5. If-Statements & Loops
6. Functions

What do you should learn in this training?

You should…

1. … get an impression about the possibilities of programming.
2. … get basic understanding for the concepts of programming.
3. … know how to structure your data.
4. … know the conventions of programming.
5. … be able to write your own (small) scripts.

Programming & Algorithms

Why we should write scripts? What is an algorithm?

Programming Languages

1. **Machine Language/ Machine code**
   - e.g. Assembler
2. **Imperative Programming**
   - e.g. Basic
3. **Procedural Programming**
   - e.g. Pascal, C
4. **Object oriented Programming**
   - e.g. Java, JavaScript, C++, C#, Python

Programming Languages

Javascript:

```javascript
function sucheDaten() {
  var daten = datenLaden();
  while(daten.vorhanden == true) {
    if (daten.korrekt == true) {
      return daten;
    }
  }
  return keineDaten();
}
```

C and C++:

```c
#include "stdio.h"
#include "daten.h"

Daten sucheDaten(void) {
  Daten daten = datenLaden();
  while(daten.vorhanden == true) {
    if (daten.korrekt == true) {
      return daten;
    }
  }
  return keineDaten();
}
```

Python:

```python
def sucheDaten():
    def daten = datenLaden()
    while daten.vorhanden == True:
      if daten.korrekt == True:
        return daten
      else:
        return keineDaten()
```

C#:

```csharp
using System;
using DatenLader;

class DatenSucher : DatenLader {
  public static object sucheDaten() {
    object daten = datenLaden();
    while(daten.vorhanden == true) {
      if (daten.korrekt == true) {
        return daten;
      }
    }
    return keineDaten();
  }
}
```

Java:

```java
//Hier ein Java-Codesnippet:
class DatenSucher extends DatenLader  {
  public Object sucheDaten() {
    Object daten = datenLaden();
    while(daten.vorhanden == true) {
      if (daten.korrekt == true) {
        return daten;
      }
    }
    return keineDaten();
  }
}
```

Visual Basic:

```vb
Imports System

Public Module Module1
  Public Function sucheDaten()
    Dim daten
    daten = datenLaden
    while daten.vorhanden = true
      if daten.korrekt = true
        return daten
      end if
    end while
    return keineDaten()
  End Function
End Module
```

Pascal:

```pascal
function sucheDaten() : object
var
  daten:object;
begin
  while daten.vorhanden = true do
  begin
    if daten.korrekt = true then exit (daten);
  end
  exit (keineDaten());
end
```

Programming Languages

Javascript:

```
function sucheDaten() {
  var daten = datenLaden();
  while(daten.vorhanden == true) {
    if (daten.korrekt == true) {
      return daten;
    }
  }
  return keineDaten();
}
```

C and C++:

```
#include "stdio.h"
#include "daten.h"

Daten sucheDaten(void) {
  Daten daten = datenLaden();
  while(daten.vorhanden == true) {
    if (daten.korrekt == true) {
      return daten;
    }
  }
  return keineDaten();
}
```

Python:

```
def sucheDaten():
  def daten = datenLaden()
  while daten.vorhanden == True:
    if daten.korrekt == True:
      return daten
    else:
      return keineDaten()
```

C#:

```
using System;
using DatenLader;

class DatenSucher : DatenLader {
  public static object sucheDaten() {
    object daten = datenLaden();
    while(daten.vorhanden == true) {
      if (daten.korrekt == true) {
        return daten;
      }
    }
    return keineDaten();
  }
}
```

Java:

```
//Hier ein Java-Codesnippet:
class DatenSucher extends DatenLader  {
  public Object sucheDaten() {
    Object daten = datenLaden();
    while(daten.vorhanden == true)
      if (daten.korrekt == true) {
        return daten;
      }
    }
    return keineDaten();
  }
}
```

Visual Basic:

```
Imports System

Public Module Module1
  Public Function sucheDaten()
    Dim daten
    daten = datenLaden
    while daten.vorhanden = true
      if daten.korrekt = true
        return daten
      end if
    end while
    return keineDaten()
  End Function
End Module
```

Pascal:

```
function sucheDaten() : object
var
  daten:object;
begin
  while daten.vorhanden = true do
  begin
    if daten.korrekt = true then exit (daten);
  end
  exit (keineDaten());
end
```

Programming Languages

Javascript:

```
function sucheDaten() {
  var daten = datenLaden();
  while(daten.vorhanden == true) {
    if (daten.korrekt == true) {
      return daten;
    }
  }
  return keineDaten();
}
```

C and C++:

```
#include "stdio.h"
#include "daten.h"

Daten sucheDaten(void) {
  Daten daten = datenLaden();
  while(daten.vorhanden == true) {
    if (daten.korrekt == true) {
      return daten;
    }
  }
  return keineDaten();
}
```

Python:

```
def sucheDaten():
  def daten = datenLaden()
  while daten.vorhanden == True:
    if daten.korrekt == True:
      return daten
    else:
      return keineDaten()
```

C#:

```
using System;
using DatenLader;

class DatenSucher : DatenLader {
  public static object sucheDaten() {
    object daten = datenLaden();
    while(daten.vorhanden == true) {
      if (daten.korrekt == true) {
        return daten;
      }
    }
    return keineDaten();
  }
}
```

Java:

```
//Hier ein Java-Codesnippet:
class DatenSucher extends DatenLader  {
  public Object sucheDaten() {
    Object daten = datenLaden();
    while(daten.vorhanden == true) {
      if (daten.korrekt == true) {
        return daten;
      }
    }
    return keineDaten();
  }
}
```

Visual Basic:

```
Imports System

Public Module Module1
  Public Function sucheDaten()
    Dim daten
    daten = datenLaden
    while daten.vorhanden = true
      if daten.korrekt = true
        return daten
      end if
    end while
    return keineDaten()
  End Function
End Module
```

Pascal:

```
function sucheDaten() : object
var
  daten:object;
begin
  while daten.vorhanden = true do
  begin
    if daten.korrekt = true then exit (daten);
  end
  exit (keineDaten());
end
```

Algorithms

algorithm

noun [C] • MATHEMATICS, COMPUTING

a set of mathematical instructions or rules that, especially if given to a computer, will help to calculate an answer to a problem:

- *Music apps use algorithms to predict the probability that fans of one particular band will like another.*

1. **Generality** The algorithm solves a lot of the same problems, a whole class of problems.
2. **Repeatability** With the same prerequisite, an algorithm always delivers the same result.
3. **Uniqueness** At each point the following step is clearly defined.
4. **Finitude/terminality** The algorithm consists of a finite number of steps and always comes to an end. The sequence of statements must also be described in a finite text.
5. **Executability** The instructions must also be clearly formulated and executable (Not for you, but for the computer).

Programming – Why?

1. It‘s simple and easy
2. Implement your own visions and ideas
3. Become independent of other programs
4. Understand the computer
5. You train your brain!

Data Types

Why do we need data types?

Data Types

Why do we need data types?

- Every Computer has only limited capacities/ resources, but e.g. the space of integers is infinitely large
  - we have to limit the space of integers

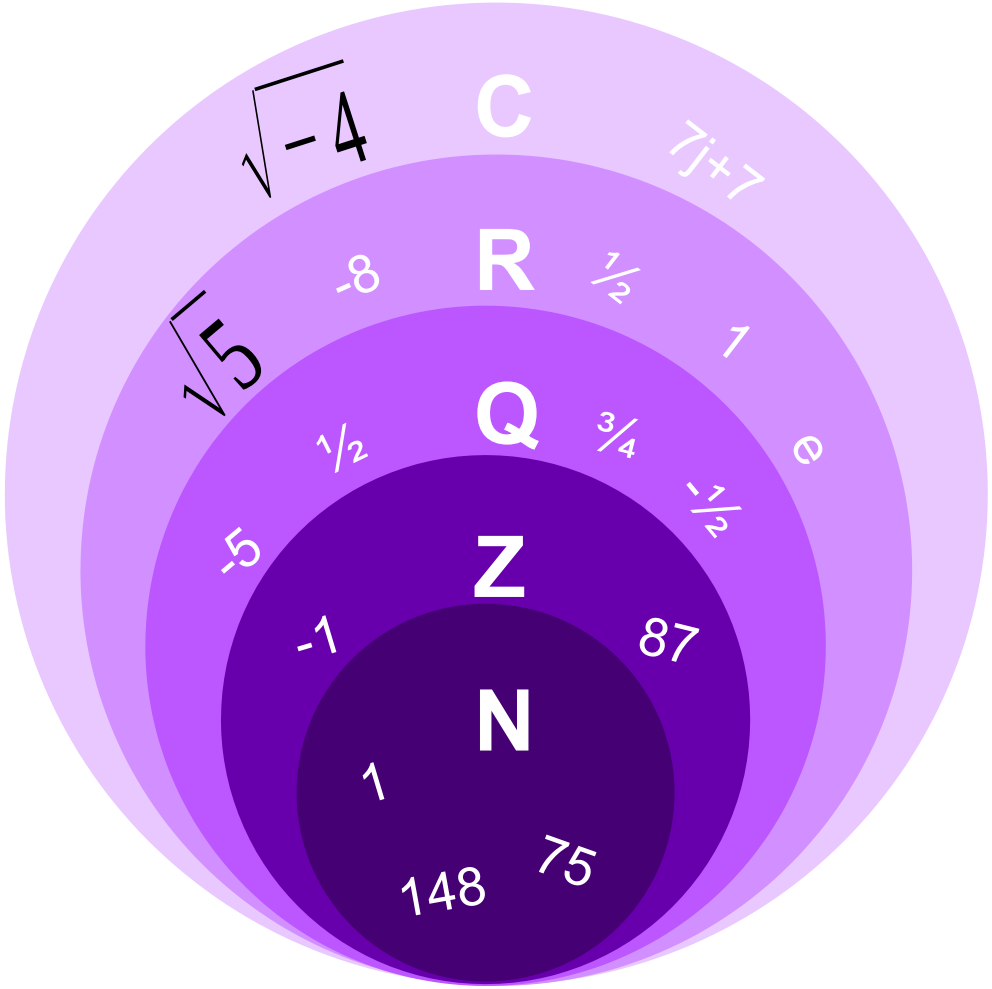
- You have to know the characteristics of the basic data types and also their advantages and disadvantages
  - choose your data type wisely and adjusted to you use case

All is about Bits & Bytes

- Bit: binary digit – positive Logic: 1 (True), 0 (False)
- Byte: four bits e.g. one byte: 1011
-

Number of **bits**	Number of **fits**
0	1
1	2
2	4
3	8
4	16
…	…
i	
…	…
32	4.294.967.296

Programming – Why?



- **C** complex numbers
- **R** real numbers
- **Q** rational numbers
- **Z** integer numbers
- **N** natural numbers

Data Types

byte	integer number (8 Bit)	- 128 to 127
short	integer number (16 Bit)	- 32 768 to 32 767
int	integer number (32 Bit)	- 2 147 483 648 to 2 147 483 647
long	integer number (64 Bit)	- 9 223 372 036 854 775 808 to 9 223 372 036 854 775 80
float	floating-point number (32 Bit)	-3.4*10^38 to +3,4*10^38
double	floating-point number (64 Bit)	-1.7*10^308 to +1.7*10^308
boolean	Only True/ False	True or False
char	character string (16 bit)	65 536 characters
string	character string (no limit*)	no limit*

* theoretical, limited by capacity of your computer

Data Types

Example:

RAM: 8 GB (Gigabyte)

8 GB = 8 000 MegaByte

8 000 MegaByte = 8 000 000 Byte = 32 000 000 Bit

short integer number (16 Bit) ☾ Two Million Variables in range of -32 768 to 32 767

int integer number (32 Bit) ☾ One Million Variables in range of -2 147 483 648 to 2 147 483 647

long integer number (64 Bit) ☾ Half a Million Variables in range of -9 223 372 036 854 775 808 to 9 223 372 036 854 775 80

float floating-point number (32 Bit) ☾ One Million Variables in range of -3.4*10^38 to +3,4*10^38

double floating-point number (64 Bit) ☾ Half a Million Variables -1.7*10^308 to +1.7*10^308

boolean Only True/ False ☾ 32 Million Variables with True or False

char character string (16 bit) ☾ Two Million Variables with 65 536 characters

Exercise 01

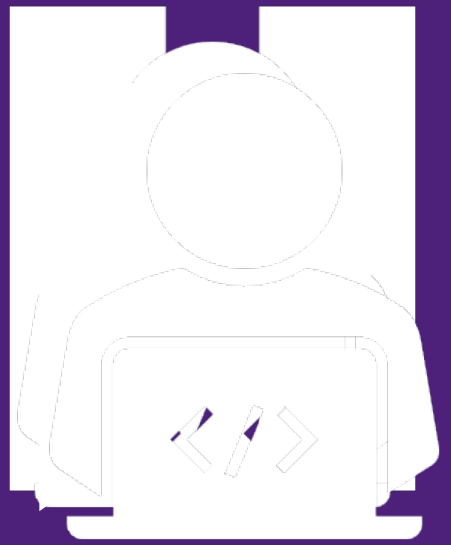
10 minutes

Data Types

What is the solution of the following calculations?

calculation	solution	data type of the solution
13 + 7 =		
’13’ + ‘7’ =		
13 + ‘7’ =		
13 * ‘7’ =		
7 * ‘s’ =		

Please think about the solution.



Exercise 01

10 minutes

Data Types

What is the solution of the following calculations?

calculation	solution	data type of the solution
13 + 7 =	20	Int
’13’ + ‘7’ =		
13 + ‘7’ =		
13 * ‘7’ =		
7 * ‘s’ =		

Please think about the solution.

Exercise 01

10 minutes

Data Types

What is the solution of the following calculations?

calculation	solution	data type of the solution
13 + 7 =	20	Int
’13’ + ‘7’ =	137	String
13 + ‘7’ =		
13 * ‘7’ =		
7 * ‘s’ =		

Please think about the solution.

Exercise 01

10 minutes

Data Types

What is the solution of the following calculations?

calculation	solution	data type of the solution
13 + 7 =	20	Int
’13’ + ‘7’ =	137	String
13 + ‘7’ =	TypeError: unsupported operand type(s) for +: 'int' and 'str'	
13 * ‘7’ =		
7 * ‘s’ =		

Please think about the solution.

Exercise 01

10 minutes

Data Types

What is the solution of the following calculations?

calculation	solution	data type of the solution
13 + 7 =	20	Int
’13’ + ‘7’ =	137	String
13 + ‘7’ =	TypeError: unsupported operand type(s) for +: 'int' and 'str'	
13 * ‘7’ =	7777777777777	String
7 * ‘s’ =		

Please think about the solution.

Exercise 01

10 minutes

Data Types

What is the solution of the following calculations?

calculation	solution	data type of the solution
13 + 7 =	20	Int
’13’ + ‘7’ =	137	String
13 + ‘7’ =	TypeError: unsupported operand type(s) for +: 'int' and 'str'	
13 * ‘7’ =	7777777777777	String
7 * ‘s’ =	sssssss	String

Please think about the solution.

Basics

Variables, Functions, Rules

Allocation & Calculation Operators

Description	Syntax	Short Operator	Long Operator
Allocation	=	x = y	
Addition	+		
Subtraction	-		
Multiplication	*		
Division	/		
Addition	+=	x += y	x = x + y
Subtraction	-=	x -= y	x = x – y
Multiplication	*=	x *= y	x = x * y
Division	/=	x /= y	x = x / y
Modulo	%=	x %= y	x = x % y
Increase by one	++	x++	x = x + 1
Decrease by one	--	x--	x = x – 1

Calculation Operators	Allocation Operators

Comparison & Logical Operators

Description	Syntax	Example
Equal	==	5 == 5 ☾ true
Is not equal	!=	5 != “4” ☾ true
Strict equal	===	5 === 5 ☾ true
Is not strict equal	!==	5 !== 4 ☾ true
Greater than	>	5 > 4 ☾ true
Less than	<	4 < 5 ☾ true
Greater than or equal	>=	5 >= 5 ☾ true
Less than or equal	<=	5 <= 5 ☾ true
AND	&& (in Python AND)	(5 == 5) && (4 != 5) ☾ true (5 === “5”) && (5 == 5) ☾ false
OR	|| (in Python OR)	(5 == 5) || (4 != 5) ☾ true (5 === “5”) || (5 == 5) ☾ true

Comparison Operators	Logical Operators

Variables

Variables

- is an abstract container for data
- the value of the data is replaceable/ changeable
- every variable reserves storage

Constants

- constants are unchangeable variables
- a change of a constant will occur an error

Declaration, Definition, Initialization

Declaration

- allocation of a name to a variable

Definition

- allocation of a memory area to a variable

Initialization

- allocation of a value to a variable

(Referencing)

- is not the same as an initialization

Declaration

```
var i;
```

Definition

```
var i=int;
```

Initialization

```
var i=0;
```

(Referencing)

```
var a = 4;
var b = 5;
var c = int;

c = b + a;
```

Recommendations for Scripting

Declaration of variables

Naming convention depends on the used programming language, but in general the following recommendations are valid:

- a name of a variables **must be unique**
- variables **must be declared** before first usage
- variables **must not contain spaces**
- the first character should be an letter (it must not be a numbers or a special character)
- its recommended to use **Camel-Case Notation** instead of underlines '_'
  - lowerCamelCase: thisIsAnExample
  - UpperCamelCase: ThisIsAnExample (also known as *Pascal Case*)

Recommendations for Scripting

Declaration of variables

Naming convention depends on the used programming language, but in general the following recommendations are valid:

- Constants should be written in Capital Letters e.g.:

```
EULERS_NUMBER = 2,718281828459045235 36
```

- do **not use reserved words** for variables (e.g.: int, double, import, var … - depends on language)
- use descriptive identifiers e.g.: `Price = ValueAddedTax *  PurchasePrice * Margin`

  instead of

```
Factor1 = F2 * F3 * F4
```

Recommendations for Scripting

Additional recommendation for writing a good script

- use spaces after an operation (=, +, -, …)
- use indentations for better readability and understanding
- Make comments: your script will be so much easier to understand for external people (and for you in future!)

```
<script>
var o=document.getElementById('o')
for(i=0;i<=6;i++){o.innerHTML+=i;};
</script>
```


```
<script>
var output = document.getElementById('output')

//count from 0 to 6 and write the number into an output
for(i = 0; i <= 6; i++)  {
                output.innerHTML += i;
                };
</script>
```

Exercise 02

10 minutes

Variables & Operators

Check if the following comparisons are *true* or *false*.

calculation	solution
3 == “3”	
3 == int(“3”)	
(3 == 4) AND (3 == 3)	
(3 == 4) OR (3 == 3)	

Data Structures

Why should/ must we structure our data?

Data Structures - Python

	Description	Usefull for…	Example
Lists	• Store collection of heterogeneous items – immutable and mutable objects	• Simple listing • Special programming techniques (Stacks, Queue, Graphs, Trees)	[item, item, item]
Dictionaries	• Collection of key-value pairs • Key: immutable objects • Value: heterogeneous items – immutable and mutable objects	• Creating loops to do the same operation on many dataset	{key1:item1, key2:item2}
Files	• Store and retrieve previously stored information		.csv, .hdf5, .xlsx
Tuples	• Tuples are data structures in which the content can not be changed after creation (no deletion, add or edit)	• Prevents data manipulation	(item, item, item)
Sets	• Collection of unique objects	• Creating lists that only hold unique values • Helpful when going through huge dataset	{item, item, item}
Numpy ND-Arrays	• Store collection of data of the same type	• Dealing with large collection of homogeneous data types: easier to use, faster and uses lesser memory than lists • Support vectorized operations • Work efficiently with large datasets with lots of empty cells	[[1. 1. 2.] [3. 5. 8.] [5. 3. 2.]]
Dataframes	• 2-dimensional labeled data structure • Columns don't have to have the same data type	• Data mining / manipulation • Labeling • Multiindexing	Like this table
Series	• One-dimensional labeled array capable of holding any data types	• Axis labels	a 1 b 2 c 3

Data Structures - General

Arrays

- Static ☾ fixed size
- fast & direct access to data
- good, if the size of the Data-Storage is known & fast access is needed
- not good, if the size is not known and you need a flexible structure
- dimension of an array is not limited

1D-Array:

1	2	3	4	5	6	7	8	…	i
43	32	18	77	4	98	17	846	…	42

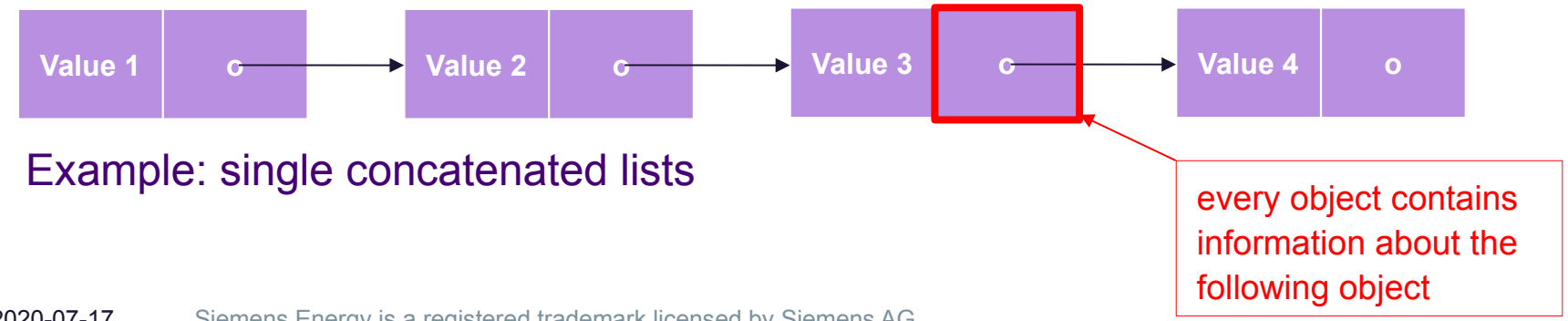
2D-Array:

	1	2	3	4	5	6	7	8	…	i
1	43	32	18	77	4	98	17	846	…	42
…	…	…	…	…	…	…	…	…	…	…
j	14	536	124	2	58	943	2	12	796	8

Data Structures - General

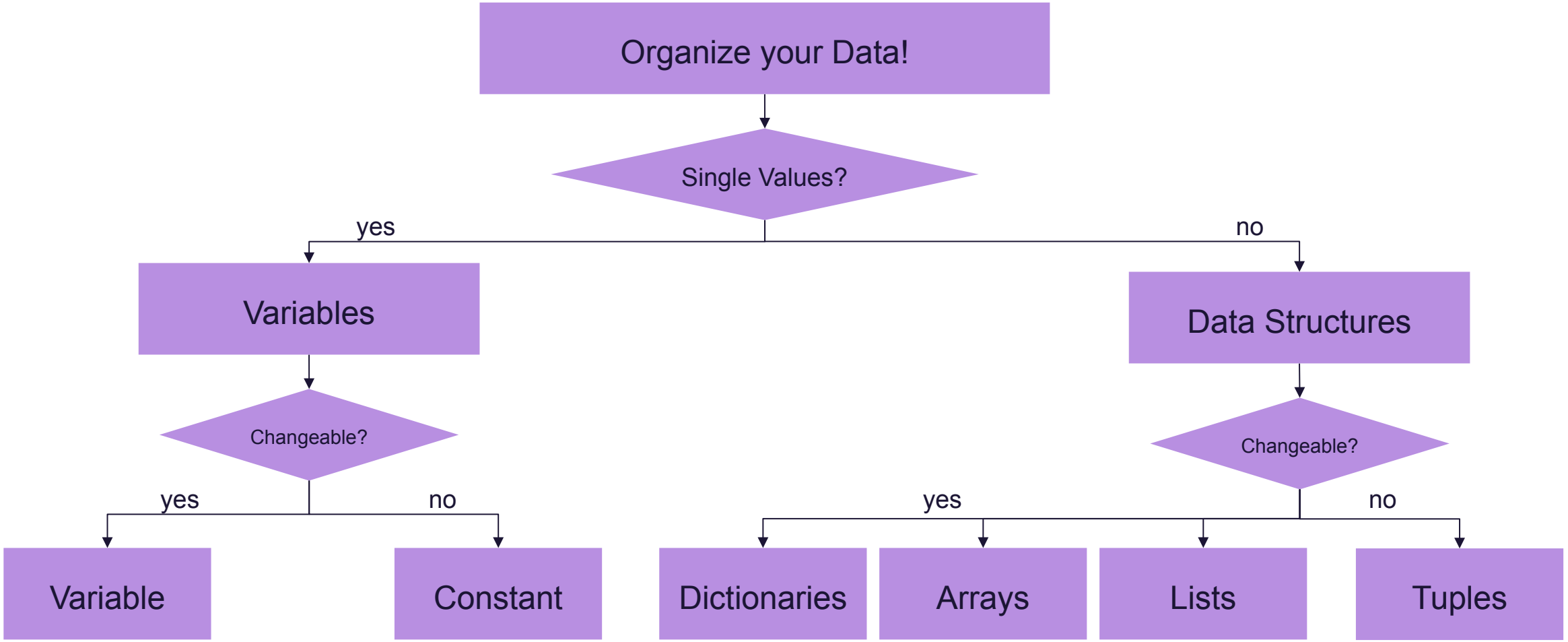
Lists

- Dynamic ☾ size is not fixed
- Concatenation of Objects
- insert/delete data at beginning, at end od in middle is possible ☾ a list can raise or shrink
- Sequential access ☾
- good, if the size of the Data-Storage is not known
- not good, if the size is known and constant and if you need a flexible storage
- **Tuple:** “the constants under the lists”



Example: single concatenated lists

Data Structures - General



Exercise 03

10 minutes

Data Structures

Use the given array to build the following string:

```
array = [['to our new', 'Siemens Energy'],['Welcome', 'company:']]

String (goal) ☹ ,Welcome to our new company: Siemens Energy‘
```

Example: array[Col][Row]

	0	1
0	to our new	Welcome
1	Siemens Energy	company:

If-Statements & Loops

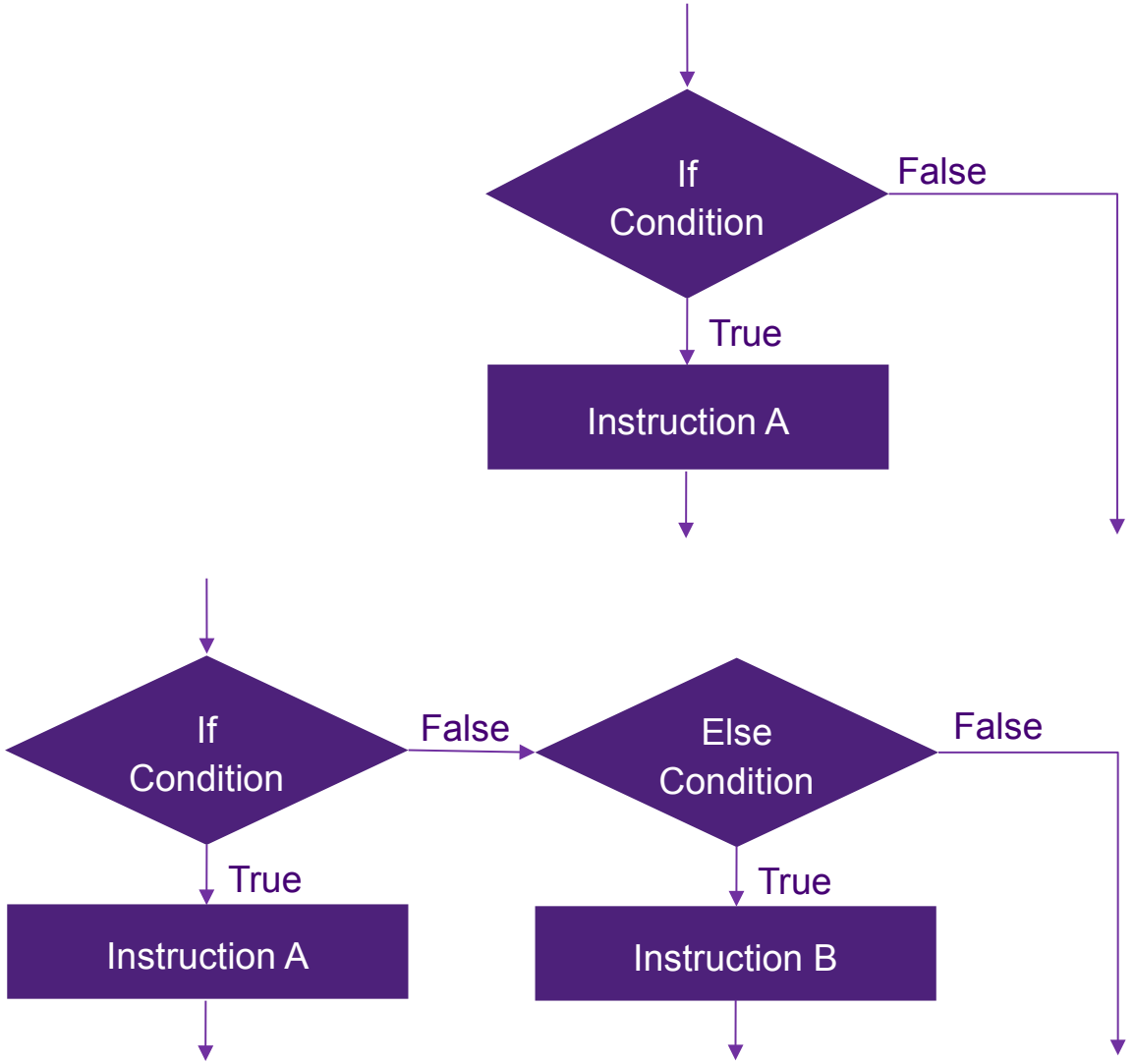
What are loops used for? Why we should use them?

If-Statements

- **If-Statement**
  - Allows to skip parts of the script, if a given condition is not true
- **If-Else-Statement**
  - Combination of if-statements

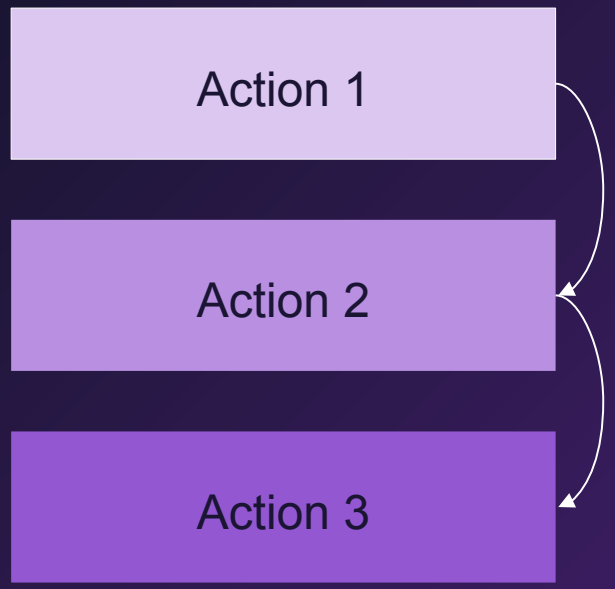
Example:

```
if(hours <= 18){
    output = 'Good Day';
}
else {
    output = 'Good Evening';
};
```

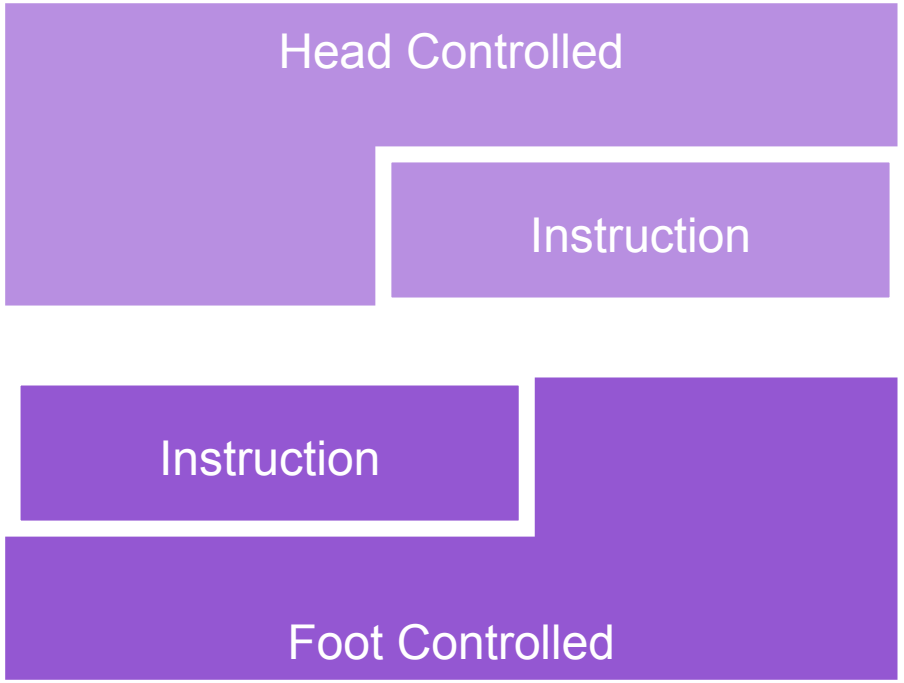


Loops

Instruction block (sequence)



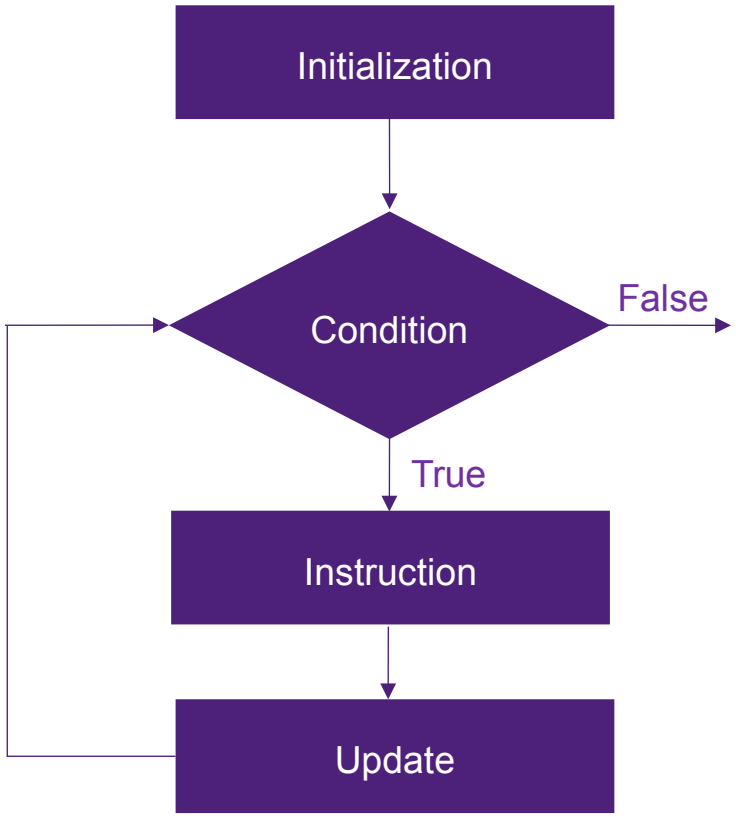
Loops



Loops: for

- Head Controlled: the condition is proven before the instruction are executed
- is used, when the number of iterations is known ☾ a number for counting is needed

First Step:	Initialization of Count-Variable
Second Step:	Proof, if the condition is true
Third Step:	If Condition is True, execute the Instructions
Fourth Step:	Update the Count-Variable
Fifth Step:	Proof if the condition is still true
Sixth Step:	If condition is not true anymore, quit the loop

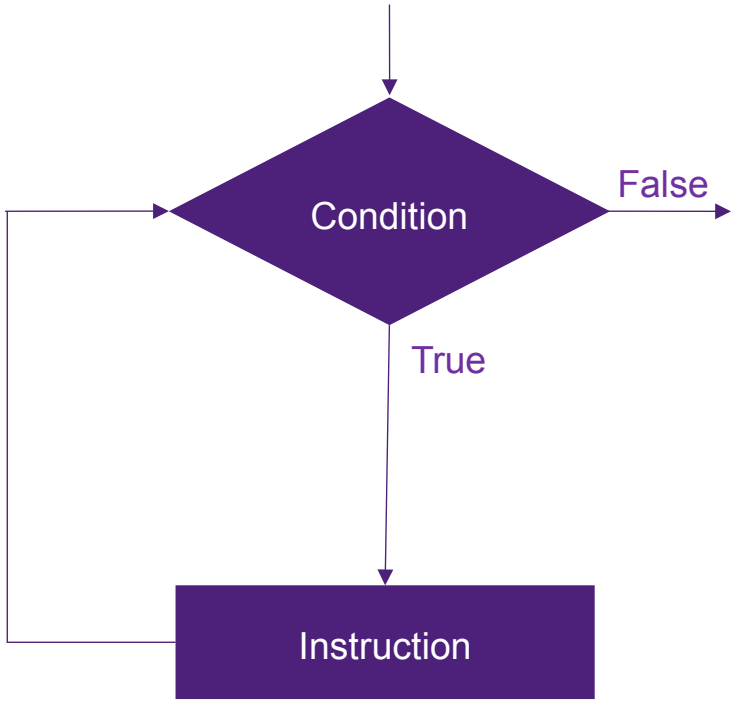


Loops: while

- Head Controlled: the condition is proven before the instruction are executed
- is used, when the number of iterations is not known e.g. if the number of iterations depends on a changing variable

First Step:	Proof if the condition is true
Second Step:	Execute the instructions
Third Step:	Proof if the condition is still true
Fourth Step:	If the condition is not true anymore, quit the loop

The Condition must be able to be false, otherwise the loop will repeat infinitely!



Loops: Continue & Break

- there are several reasons to end a loop prematurely
- there are two possibilities for end a loop:
  - Continue: end the iteration and begin the loop again
  - Break: end the iteration and terminate the loop

Continue:

```
i=0
while(i < 6):
    i = i+1
    if(i == 4):
        continue
    print(i)

>>> 1
>>> 2
>>> 3
>>> 5
>>> 6
```

Break:

```
i=0
while(i < 6):
    i = i+1
    if(i == 4):
        break
    print(i)

>>> 1
>>> 2
>>> 3
```

Exercise 04

15 minutes

Loops

Check out the differences between for-Loop and while-Loop and between continue and break.

- Write two different Loops:

  - First you should write a for-Loop that counts from 0 to 9.

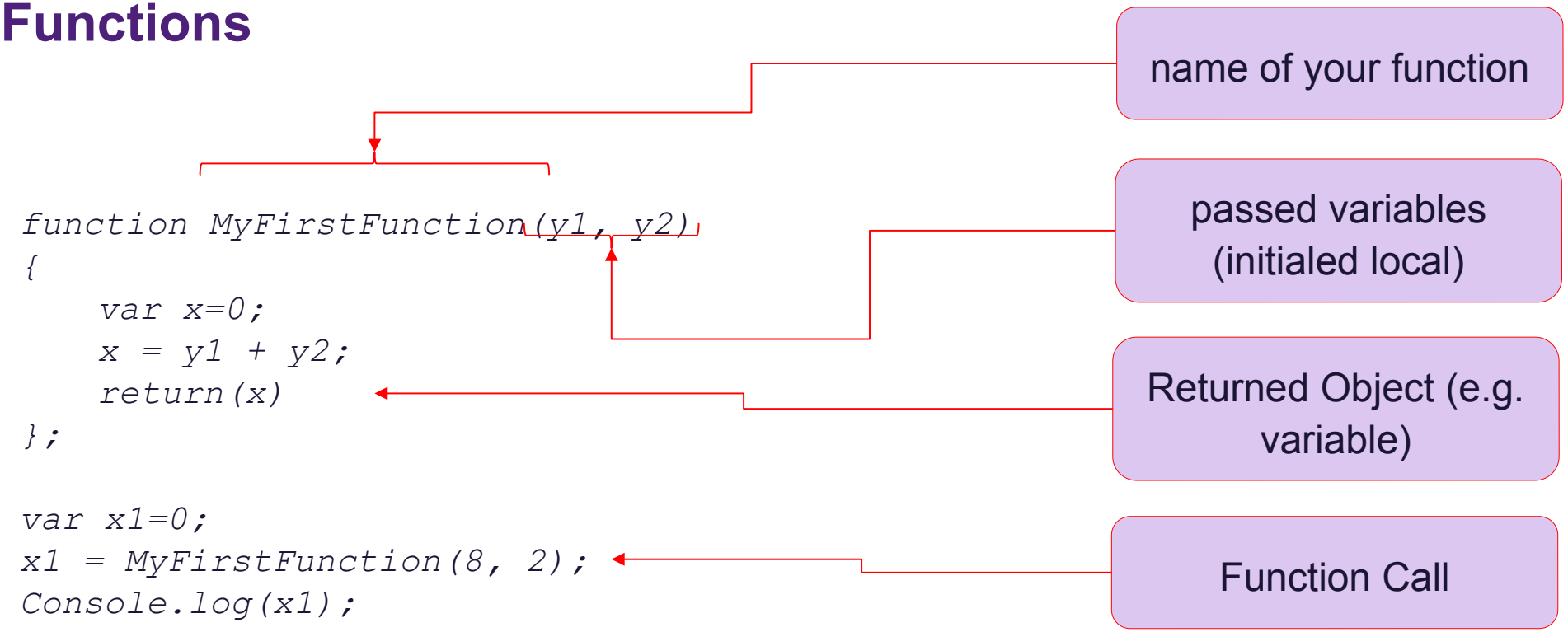
  - The second loop should be a while loop, counting from 0 to the length of the array named exampleArray_01

- Have a look on the last two loops (while-loops). Try them out and clearify, what the difference is between using **'continue'** and using **'break'**.

Functions

The smart way to writing good scripts.

Functions




```
function MyFirstFunction(y1, y2)
{
    var x=0;
    x = y1 + y2;
    return(x)
};

var x1=0;
x1 = MyFirstFunction(8, 2);
Console.log(x1);
>> 10
```

- are used, if (complex) instructions are needed an different places in script
- allows you to shorten your script and keep it clear

Functions

Initialization

```
def myFirstFunction(a, b, operation="multiplication"):
    """
    ##### Description #####

    a: float
    b: float
    operation: string, please choose between "addition", "subtraction", "multiplication" or "division"
    """
    assert (type(a) == float) | (type(a) == int), "Error: a must be a float or int"
    assert (type(b) == float) | (type(a) == int), "Error: b must be a float or int"
    assert (operation == "addition") | (operation == "subtraction") | (operation == "multiplication") | (operation == "division"),
      "Error: please choose between 'addition', 'subtraction', 'multiplication' or 'division'"

    if(operation == "addition"):
        c = a + b

    if(operation == "subtraction"):
        c = a - b

    if(operation == "multiplication"):
        c = a * b

    if(operation == "division"):
        c = a / b

    return c
```

Functions

Description

```
def myFirstFunction(a, b, operation="multiplication"):
    """
    ##### Description #####

    a: float
    b: float
    operation: string, please choose between "addition", "subtraction", "multiplication" or "division"
    """
    assert (type(a) == float) | (type(a) == int), "Error: a must be a float or int"
    assert (type(b) == float) | (type(a) == int), "Error: b must be a float or int"
    assert (operation == "addition") | (operation == "subtraction") | (operation == "multiplication") | (operation == "division"),
      "Error: please choose between 'addition', 'subtraction', 'multiplication' or 'division'"

    if(operation == "addition"):
        c = a + b

    if(operation == "subtraction"):
        c = a - b

    if(operation == "multiplication"):
        c = a * b

    if(operation == "division"):
        c = a / b

    return c
```

Functions

```
def myFirstFunction(a, b, operation="multiplication"):
    """
    ##### Description #####

    a: float
    b: float
    operation: string, please choose between "addition", "subtraction", "multiplication" or "division"
    """
    assert (type(a) == float) | (type(a) == int), "Error: a must be a float or int"
    assert (type(b) == float) | (type(a) == int), "Error: b must be a float or int"
    assert (operation == "addition") | (operation == "subtraction") | (operation == "multiplication") | (operation == "division"),
     "Error: please choose between 'addition', 'subtraction', 'multiplication' or 'division'"

    if(operation == "addition"):
        c = a + b

    if(operation == "subtraction"):
        c = a - b

    if(operation == "multiplication"):
        c = a * b

    if(operation == "division"):
        c = a / b

    return c
```

Error Messages

Functions

```
def myFirstFunction(a, b, operation="multiplication"):
    """
    ##### Description #####

    a: float
    b: float
    operation: string, please choose between "addition", "subtraction", "multiplication" or "division"
    """
    assert (type(a) == float) | (type(a) == int), "Error: a must be a float or int"
    assert (type(b) == float) | (type(a) == int), "Error: b must be a float or int"
    assert (operation == "addition") | (operation == "subtraction") | (operation == "multiplication") | (operation == "division"),
     "Error: please choose between 'addition', 'subtraction', 'multiplication' or 'division'"
```

Instructions

```
if(operation == "addition"):
    c = a + b

if(operation == "subtraction"):
    c = a - b

if(operation == "multiplication"):
    c = a * b

if(operation == "division"):
    c = a / b

return c
```

Exercise 05

20 minutes

Functions

Please write your own first little Function.

Your function should compare two values and return, if the first value is smaller/ bigger/ equal to the second value.

e.g.: the following code should return "21 is smaller than 3"

```
result = compareFunction(21,3)

print(result)

>>> 21 is smaller than 3
```

Attention: You have to parse the integer value of a and b into a string, so that you can stick it to the string "is smaller than"/ "is bigger than"/ "is equal to"

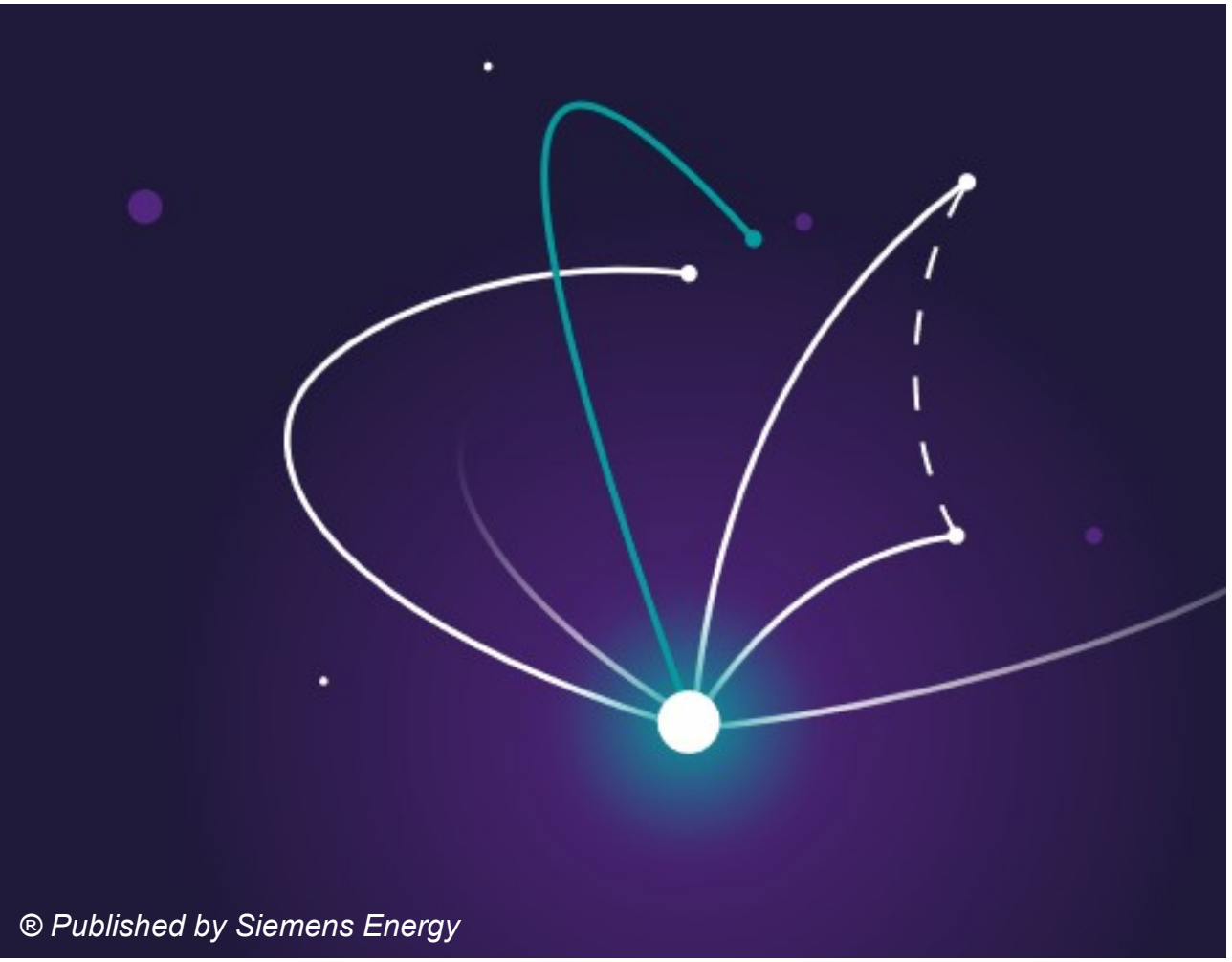
The n-Commandments

1. Beautiful is better than ugly.
2. Explicit is better than implicit.
3. Simple is better than complex.
4. Complex is better than complicated.
5. Flat is better than nested.
6. Sparse is better than dense.
7. Readability counts.
8. Special cases aren't special enough to break the rules.
9. Errors should never pass silently.
10. In the face of ambiguity, refuse the temptation to guess.
11. There should be one -- and preferably only one -- obvious way to do it.
12. Now is better than never.
13. If the implementation is hard to explain, it's a bad idea/ If the implementation is easy to explain, it may be a good idea.

https://www.w3schools.com/python/

Thank you very much for your attention.

Contact



Nicolas Schneider

Dual Student

SE G QPI QM P

Mellinghofer Straße 55
45468 Mülheim an der Ruhr
Germany

Phone: +49 173 4278041

nicolas.schneider@siemens.com

siemens-energy.com